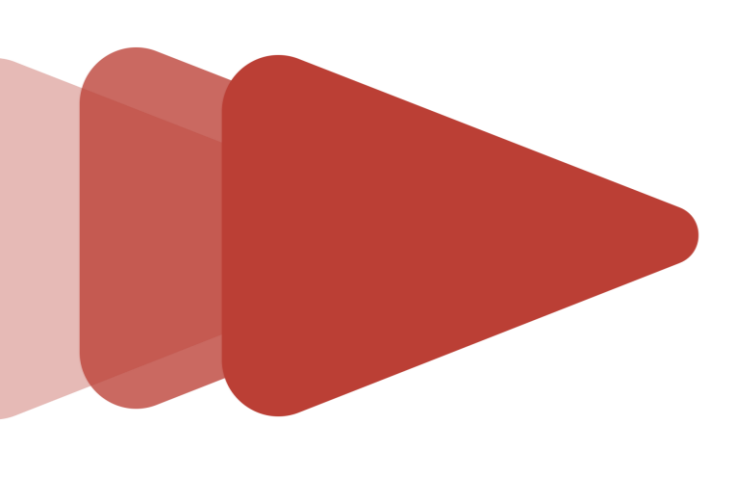




Estándar Tecnológico Plataforma de Gestión de APIs

<<Número de Documento>>

Gestión de Datos y Arquitectura Tecnológica
Departamento de Planificación TI y Gestión de Información - Datos
Dirección Nacional Tecnología

Fecha de Actualización: Junio 2026
Versión: 1.0.0

Cláusula de Responsabilidad

Los firmantes de este documento, en representación de sus unidades administrativas a las que pertenecen, aceptan y aprueban el presente documento y a su vez se comprometen a respaldar su cumplimiento, hacer difusión del mismo en sus respectivas unidades y a sugerir mejoras para versiones futuras en el caso de que se las identifiquen.

Fecha de Vigencia dd/05/2026	Dirección Nacional de Tecnología	VERSIÓN 1.0.0	PÁGINAS 24
Estándar de Plataforma de Gestión de APIs			
	CARGO	FIRMA	
APROBADO POR:	Paulina Correa A. DIRECTOR NACIONAL DE TECNOLOGÍA		
REVISADO POR:	Johanna Chagñay C. JEFE NACIONAL DEPARTAMENTO PLANIFICACIÓN TI Y GESTIÓN INFORMACIÓN - DATOS		
	Edison Cayambe Ch. COORDINADOR NACIONAL GESTIÓN DE DATOS Y ARQUITECTURA TECNOLÓGICA		
ELABORADO POR:	Santiago Simbaña H. EXPERTO NACIONAL DE ARQUITECTURA DE SOLUCIONES TECNOLÓGICAS		

INFORMACIÓN DEL DOCUMENTO

Resumen:	Estándar institucional para la gobernabilidad del ciclo de vida de APIs REST internas y externas, mediante una plataforma de gestión agnóstica de herramienta. Define reglas de gobierno, seguridad, operación y calidad aplicables a toda API publicada.
Alcance:	Aplica a todas las APIs REST/HTTP internas y externas gestionadas por la institución. NO contempla estándares de desarrollo frontend, backend, DevSecOps ni contenedores (regulados por estándares específicos).
Audiencia:	Equipos de Arquitectura, Plataforma, Desarrollo, Seguridad, QA y Operaciones que participan en el ciclo de vida de APIs institucionales.

REVISIONES AL DOCUMENTO

Versión -Fecha	Autor	Descripción
1.0.0 / 2026-05-dd	Santiago Simbaña	Emisión Inicial

Enmienda del documento

El Departamento de Planificación TI y Gestión de Información - Datos tiene la potestad de actualizar el contenido de los Anexos cuando lo considere necesario y no requerirá recolección de firmas para su aprobación. Sin embargo, se comunicarán a las Coordinaciones y Departamentos involucrados los cambios a realizarse, quienes podrán indicar sus observaciones al respecto en los siguientes ocho días laborables, luego de lo cual se considerarán aceptadas, y se procederá a su publicación.

La aplicación de las modificaciones al estándar en proyectos con desarrollos terminados o en curso será consensuado con el Departamento de Desarrollo, con el fin de evaluar el impacto (positivo o negativo) que tenga sobre el producto o sobre la calidad del mismo, adicionalmente se debe informar de los acuerdos a las Coordinaciones de Control y Aseguramiento de Calidad (QA) y Centro de Cómputo.

Ante cualquier sugerencia de mejora al documento debe ser comunicado al Departamento de Planificación TI y Gestión de Información - Datos para su análisis, evaluación de factibilidad y posterior actualización de la documentación en caso de considerarse necesario.

En caso de ser necesario emitir un lineamiento técnico, aclaración o alcance al presente documento, este podrá ser emitido mediante memorando suscrito por el Jefe Departamental de Planificación TI y Gestión de Información - Datos en coordinación con la Dirección Nacional de Tecnología; y se mantendrá como una disposición vigente hasta su incorporación en la siguiente actualización del documento.

Contenido

1. Introducción	5
1.1. Objetivos	5
2. Estándares Generales	6
3. Plataforma de Gestión de APIs	7
3.1. API Gateway	7
4. Gobierno y Ciclo de Vida	9
5. Gobernanza REST	9
6. Glosario	11
a. Acrónimos	11
b. Definiciones	13
7. Bibliografía	15
8. Anexos	16
Anexo A: Catálogo API - YAML de registro	16
Anexo B: Registro de Consumidor – YAML Registro	17
Anexo C: Planes de Uso (Rate Limit y cuota)– YAML	18
Anexo D: Configuración Gateway por API – YAML	19
Anexo E. Configuración CORS por API – YAML	20

1.Introducción

El Servicio de Rentas Internas atraviesa una transición tecnológica estructural: la migración de sistemas monolíticos hacia microservicios, APIs RESTful y contenedores orquestados con Kubernetes genera un crecimiento sostenido en el número de interfaces expuestas, los puntos de integración entre sistemas y los equipos involucrados en su construcción y operación. Sin un modelo de gobierno común, este crecimiento deriva en registro no controlado de servicios, consumidores sin credenciales únicas, integraciones frágiles sin contrato versionado, incidentes difíciles de diagnosticar por ausencia de trazabilidad, y ausencia de claridad sobre quién consume qué API, bajo qué condiciones y con qué vigencia.

Este estándar establece las reglas mínimas, verificables y aplicables mediante pipeline automatizado, para administrar las APIs institucionales a través de la Plataforma de Gestión de APIs. El documento regula el registro, clasificación, publicación, consumo, versionamiento, deprecación, retiro y gestión de excepciones de APIs REST/HTTP.

El estándar se alinea explícitamente con el Estándar DevSecOps, ninguna regla de este documento duplica lo regulado en el documento referenciado.

1.1. Objetivos

- Establecer las reglas mínimas para gobernar las APIs institucionales del SRI.
- Definir responsabilidades claras entre los equipos de Arquitectura, Desarrollo, QA, Seguridad y Operaciones.
- Proveer criterios de aceptación verificables para cada control, de modo que QA pueda validarlos sin ambigüedad.
- Alinear el gobierno de APIs con el Estándar DevSecOps ARE2604 v1.0.0 vigente en la institución.
- Servir como referencia única para el registro, publicación, consumo, deprecación y retiro de APIs en el SRI.

2. Estándares Generales

Regla: Toda API REST/HTTP debe ser registrada en el catálogo de la plataforma de API Management antes de su integración y despliegue en el ambiente correspondiente cuando el archivo YAML (Anexo A) cumpla con la información requerida.

Utilidad: Evita la proliferación de APIs fuera de gobierno, mejora la trazabilidad institucional y centraliza la administración del ciclo de vida.

Regla: Todo deploy de una API al ambiente superior a desarrollo debe ejecutarse mediante el pipeline definido en .gitlab-ci.yml. El pipeline contiene stages bloqueantes. Si cualquier stage falla, el deploy no se ejecuta.

- Lint: El openapi.yml cumpla con el esquema de OpenAPI 3.1.
- Contract-test: ejecuta pruebas de la colección de datos con test positivo (token válido→2xx) y negativo (token inválido→401)
- Security-scan: OWASP API Security Top 10 sobre el contrato OpenAPI

Utilidad: Prevención sistemática de defectos. Pipeline bloquea si el OpenAPI es inválido.

Regla: Ninguna API en estado published puede pasar a estado retired sin transitar primero por deprecated. El plazo mínimo en estado deprecated antes del retiro y según criticidad registrada en el catálogo es:

- Baja: mínimo 30 días calendario antes del retiro.
- Media: mínimo 60 días calendario antes del retiro.
- Alta: mínimo 90 días calendario antes del retiro.

Utilidad: Evita interrupciones no planificadas en sistemas consumidores.

Regla: Toda API publicada en ambientes de preproducción y producción debe utilizar HTTPS con TLS 1.2 como versión mínima. Queda prohibida la exposición de APIs sin cifrado en tránsito.

Utilidad: Garantiza la confidencialidad e integridad de los datos en tránsito, previniendo ataques de interceptación (man-in-the-middle). Alineado con los valores baseline de TLS definidos en el Anexo B.

Regla: Toda API publicada en producción debe implementar al menos un mecanismo de autenticación (OIDC, OAuth 2.0, mTLS u otro aprobado por Seguridad). Queda prohibida la publicación de APIs sin autenticación en producción, salvo APIs públicas informativas previamente aprobadas por el Departamento de Asuntos Internos y el usuario funcional con vigencia máxima de 365 días.

Utilidad: Asegura que solo consumidores identificados y autorizados accedan a los recursos de la API, reduciendo la superficie de ataque. La ausencia de autenticación corresponde al riesgo API1 (Broken Object Level Authorization) y API2 (Broken Authentication) del OWASP API Security Top 10 2023.

Regla: Queda prohibido transmitir credenciales, tokens de acceso, API keys o secretos en parámetros de URL (query string). Estos valores deben transportarse exclusivamente mediante cabeceras HTTP (Authorization u otras cabeceras personalizadas) sobre una conexión HTTPS.

Utilidad: Evita la exposición de credenciales en logs de servidores, proxies, caché de navegadores e historial HTTP, reduciendo el riesgo de filtración involuntaria.

3. Plataforma de Gestión de APIs

La plataforma es el conjunto de capacidades que gobierna el ciclo de vida de las APIs. El estándar es agnóstico de herramienta; por tanto, las reglas aplican independientemente del producto utilizado, siempre que este proporcione las capacidades.

Regla: Toda API publicada en producción debe haber pasado por los ambientes dev / testintg/ preproducción / producción. El pipeline bloquea cualquier intento de deploy a producción si no existe un deploy exitoso previo en preproducción con el mismo hash de artefacto.

Utilidad: Permite controlar la promoción de configuración de API Management antes de exponer APIs productivas.

3.1. API Gateway

El Gateway es el punto único de ejecución de políticas para la siguiente clasificación de APIs .

- API interna: API consumida por sistemas o aplicaciones institucionales dentro de la red o dominio tecnológico institucional.
- API externa: API expuesta a ciudadanos, contribuyentes, entidades externas o canales digitales institucionales.
- API partner / B2B: API expuesta a consumidores específicos bajo convenio, acuerdo de integración o autorización formal.

Las siguientes reglas aplican a toda API publicada a través del Gateway:

Regla: Toda API debe tener un plan de uso declarado mediante el YAML (Anexo C), con lo cual el Gateway aplica el rate limit y la cuota declarados en el YAML. La plataforma rechaza el deploy de una API sin plan asignado.

Utilidad: Protección anti-abuso y estabilidad del backend.

Regla: Para toda API cuyo YAML de registro (Anexo B) contiene al menos un consumer con client_type: browser, el Gateway debe tener configurada una política CORS con lista de orígenes explícita. El campo allowed_origins debe contener URLs completas con protocolo y dominio, el valor * (wildcard) está prohibido.

El campo client_type en el YAML del consumidor determina si se aplica CORS:

- client_type: browser → CORS requerido con allowed_origins explícita
- client_type: server → CORS no aplica
- client_type: mobile → CORS no aplica

Si el campo no está declarado, el gate lo trata como error y bloquea el deploy.

Utilidad: Prevención de exposición accidental a orígenes no autorizados.

Regla: El Gateway debe propagar en cada solicitud al backend los siguientes headers de identidad:

- X-Consumer-id: consumerId del catálogo
- X-Consumer-Username: nombre del consumidor.
- X-Scopes: Scopes autorizados en el token.
- X-Correlation-Id: UUID v4 generado por el Gateway

Utilidad: Observabilidad y auditoría sin replicar lógica de autorización en cada servicio.

Regla: La configuración del Gateway (rutas, planes, scopes, rate limits, CORS, allowlists) debe mantenerse como código en repositorio versionado. No se aplican cambios manuales en producción.

```

api-platform-gateway/      # repositorio GitLab centralizado
├── .gitlab-ci.yml          # pipeline de despliegue de configuración
├── api-catalog/           # catálogo (fuente de verdad del inventario)
│   ├── {dominio1}/
│   │   ├── {dominio1}-rest.yaml    # Anexo C
│   │   └── {dominio2}/
│   │       └── {dominio2}-rest.yaml
│   └── routes/              # configuración por API (una carpeta por productId)
│       ├── {dominio1}-rest/
│       │   ├── route.yaml         # URL del backend, timeouts, retries
│       │   ├── cors.yaml          # política CORS
│       │   ├── headers-policy.yaml # transformación de headers (R-14)
│       │   └── rate-limit.yaml     # enlaza al planId del Anexo B
│       └── {dominio2}-rest/
├── plans/                 # Planes de uso (Anexo B)
│   ├── plan-externo-basico.yaml
│   └── plan-partner.yaml
├── consumers/            # Consumidores registrados
│   ├── banco-amarillo-sistemas.yaml #
│   └── aplicativo01-sistemas.yaml    #
└── env/
    ├── testing.yaml        # endpoints del Gateway en Testing
    ├── preprod.yaml
    ├── prod.yaml
    └── dev.yaml

```

Utilidad: Trazabilidad, auditoría y repetibilidad. Elimina configuraciones fantasmas.

Ejemplo: Repositorio: api-platform-{dominio}-gateway/ con estructura products/, plans/, routes/, policies/, env/.

Regla: Todas las APIs deben agruparse por dominio/capacidad de negocio (no por tecnología). El Catálogo debe reflejar esa agrupación.

Utilidad: Mejora interoperabilidad, descubrimiento y gestión por consumidores.

Regla: El Gateway debe validar las solicitudes entrantes contra el esquema del contrato OpenAPI publicado (tipos de datos, campos obligatorios, formatos). Las solicitudes que no cumplan el esquema deben ser rechazadas con HTTP 400 antes de enrutarse al backend.

Utilidad: Previene ataques de inyección y manipulación de parámetros; descarga al backend de la validación de entradas malformadas.

Regla: Para APIs externas de criticidad Alta o Media, debe habilitarse un WAF (Web Application Firewall) u otro mecanismo equivalente de protección en capa 7 que detecte y bloquee patrones de ataque conocidos (inyección, fuzzing, escaneos automatizados). La activación del WAF debe registrarse en el catálogo de la API.

Utilidad: Agrega una capa de defensa en profundidad ante amenazas de capa de aplicación, complementando los controles de autenticación, autorización y rate limit ya definidos.

Regla: El Gateway debe validar al menos los siguientes claims del token JWT antes de enrutar la solicitud: exp (token no expirado), iss (emisor pertenece al IdP institucional) y aud (audiencia corresponde a la API destino). Tokens que no superen estas validaciones deben ser rechazados con HTTP 401.

Utilidad: Previene el uso de tokens expirados, emitidos por fuentes no confiables o destinados a otro servicio, reduciendo el riesgo de acceso no autorizado por reutilización o falsificación de tokens.

Regla: La plataforma debe registrar como eventos de seguridad auditables, accesibles para el equipo de Seguridad: intentos de autenticación fallidos, violaciones de rate limit sostenidas, rechazos de

CORS, solicitudes bloqueadas por WAF, accesos con tokens revocados o expirados, y cambios críticos de configuración en Gateway.

Utilidad: Habilita la detección temprana de ataques, abusos de API y configuraciones incorrectas, y facilita la respuesta a incidentes con evidencia trazable.

4. Gobierno y Ciclo de Vida

Regla: La gestión de APIs en la plataforma de API Management debe contar con roles mínimos definidos para administrar, aprobar, validar y operar la publicación y consumo de APIs.

La siguiente tabla establece los roles mínimos requeridos para la gestión de APIs en la plataforma:

Rol	Definición	Perfil
API Owner Técnico	Responsable funcional y técnico de la API. Inicia registro, coordina cambios y deprecación.	Conocimiento capacidad técnica.
API Owner Funcional	Responsable funcional y suministra la información del negocio que la API requiere.	Conocimiento funcional del negocio
Desarrollo	Implementa el contrato OpenAPI, escribe el pipeline CI/CD, construye los gates afines.	Técnico.
Control de Calidad	Verifica cumplimiento de reglas.	Técnico.
Seguridad	Revisa mTLS, allowlists y clasificación de datos sensibles	Técnico en seguridad.
Operaciones	Administra la plataforma, monitoreo PROD	Técnico operativo. Acceso de administrador a la plataforma.
Plataforma API Mgt	Rol Operaciones especializado: administra el API Gateway, catálogo, planes y consumidores	Técnico especialista en la herramienta de API Management institucional.

Utilidad: Estos roles definen responsabilidades mínimas para la administración y gobierno de APIs, evitando ambigüedad en la aprobación, publicación, operación, control de consumidores y atención de incidentes.

Regla: Toda API debe transitar por estados controlados en la plataforma:

Estado	Descripción
Draft	API registrada, aún no publicada en producción
Published	API activa con consumidores autorizados
Deprecated	API marcada para retiro; no acepta nuevos consumidores
Retired	API bloqueada por plataforma; devuelve 410

Utilidad: Permite gobernar formalmente la vida de una API desde su identificación hasta su baja.

5. Gobernanza REST

Regla: Los scopes deben seguir la convención `api:{productId}:{accion}` y asignarse por mínimo privilegio. Solo los estrictamente necesarios para el caso de uso.

Utilidad: Reduce impacto ante credenciales comprometidas.

Ejemplo: api:anexos-consulta:read (solo lectura, no se concede :admin sin justificación).

Regla: La plataforma en el Gateway aplica los siguientes valores baseline a toda API, salvo excepción justificada.

Control	Criticidad Alta	Criticidad Media	Criticidad Baja
Rate limit (req/min/consumidor)	50	100	200
Quota diaria	5.000	20.000	Sin límite
Timeout conexión	5s	10s	15s
Timeout respuesta	15s	30s	60s
Tamaño máximo request	1 MB	5 MB	10 MB
TLS mínimo	TLS 1.3 + mTLS	TLS 1.2	TLS 1.2
SLO disponibilidad	≥ 99,5%	≥ 99%	Best effort
Ventana deprecación	90 días	60 días	30 días

Utilidad: Previene agotamiento de recursos, DoS por payload grande y latencias indefinidas.

Regla: Antes de habilitar en ambiente de producción una API REST, debe verificarse en ambiente de preproducción y registrarse como evidencia las siguientes comprobaciones:

#	Verificación requerida	Criterio de aceptación
1	OpenAPI versionado y referenciado (tag inmutable) en Catálogo	Catálogo apunta a tag inmutable en repositorio
2	Catálogo completo y estado Published	Todos los campos obligatorios completos
3	Autenticación OIDC activa en Gateway	Token inválido → 401 / Token válido → 2xx
4	Autorización por scopes aplicada	Con scope → permitido / Sin scope → 403
5	Rate limit activo	Exceso → 429 con Retry-After
6	Timeouts y límites de payload activos	Request excesivo → 413 / Timeout → 504
7	Observabilidad mínima activa	Logs + métricas + correlation-id propagado
8	HTTPS/TLS activo en Gateway	HTTPS activo; versión TLS ≥ 1.2 verificada; sin endpoints HTTP en PROD
9	Validación de esquema OpenAPI activa	Solicitud con campo inválido u obligatorio ausente → 400
10	Errores normalizados; sin stack traces en respuestas al consumidor	Error de backend → respuesta genérica HTTP sin información interna

6. Glosario

a. Acrónimos

Acrónimo	Significado
AO	API Owner — responsable funcional y técnico de una API
API	Application Programming Interface — Interfaz de programación de aplicaciones
APIM	API Management — Plataforma de gestión del ciclo de vida de APIs
ARE2604	Estándar DevSecOps institucional SRI v1.0.0, febrero 2026
ARQ	Arquitectura de Soluciones — rol institucional que define el estándar y los Anexos
AsyncAPI	AsyncAPI Specification — estándar de contrato para APIs orientadas a eventos
AuthN	Authentication — proceso de verificación de identidad
AuthZ	Authorization — proceso de verificación de permisos y accesos
B2B	Business-to-Business — integración entre sistemas de organizaciones distintas
CA	Certificate Authority — autoridad certificadora de certificados digitales
CI/CD	Continuous Integration / Continuous Delivery — integración y entrega continua
CORS	Cross-Origin Resource Sharing — política de intercambio de recursos entre orígenes
CVE	Common Vulnerabilities and Exposures — identificador estándar de vulnerabilidades
DAST	Dynamic Application Security Testing — análisis dinámico de seguridad
DDD	Domain-Driven Design — diseño orientado al dominio
DEV	Desarrollo — equipo que implementa contratos OpenAPI y pipelines CI/CD
DLQ	Dead Letter Queue — cola de mensajes fallidos en arquitecturas de eventos
DNT	Dirección Nacional de Tecnología — SRI
DoS	Denial of Service — ataque de denegación de servicio
DX	Developer Experience — experiencia del desarrollador consumidor de APIs
FQDN	Fully Qualified Domain Name — nombre de dominio completamente calificado
HTTP	HyperText Transfer Protocol — protocolo de transferencia de hipertexto
HTTPS	HTTP Secure — HTTP sobre TLS
IaC	Infrastructure as Code — infraestructura definida y versionada como código
IdP	Identity Provider — proveedor de identidad; en SRI: Red Hat SSO / Keycloak
IDP	Identity Provider — sinónimo de IdP
IETF	Internet Engineering Task Force — organismo que publica estándares RFC
JWT	JSON Web Token — token de autenticación/autorización firmado digitalmente
M2M	Machine-to-Machine — comunicación sistema a sistema sin intervención humana
MR	Merge Request — solicitud de fusión de código en GitLab
mTLS	Mutual TLS — autenticación mutua mediante certificados en ambos extremos
MTTR	Mean Time To Recovery — tiempo medio de recuperación ante incidentes
OAS	OpenAPI Specification — estándar de contrato para APIs REST (antes Swagger)
OAuth 2.0	OAuth 2.0 Authorization Framework (RFC 6749) — marco de autorización con tokens
OIDC	OpenID Connect — capa de identidad sobre OAuth 2.0, base del SSO institucional

OpenAPI	OpenAPI Specification — estándar YAML/JSON para describir APIs HTTP/REST (v3.1)
OPS	Operaciones — equipo que administra la plataforma y monitorea PROD
OTel	OpenTelemetry — marco de observabilidad: trazas, métricas y logs
OWASP	Open Web Application Security Project — organización de seguridad de aplicaciones
PII	Personally Identifiable Information — información de identificación personal
PKI	Public Key Infrastructure — infraestructura de clave pública para certificados
PLAT	Plataforma API Management — rol operativo especializado en el Gateway institucional
PREPROD	Preproducción — ambiente de configuración idéntica a PROD para validación final
PROD	Producción — ambiente productivo con consumidores reales
QA	Quality Assurance / Control de Calidad — verifica cumplimiento de reglas del estándar
RACI	Responsible / Accountable / Consulted / Informed — matriz de responsabilidades
REST	Representational State Transfer — estilo arquitectónico para APIs HTTP
RFC	Request for Comments — documentos de estándares de internet (IETF)
RPS	Requests Per Second — solicitudes por segundo
SAST	Static Application Security Testing — análisis estático de seguridad del código
SCA	Software Composition Analysis — análisis de dependencias y licencias
SEC	Seguridad — equipo de seguridad informática; revisa mTLS, allowlists y datos sensibles
SemVer	Semantic Versioning — versionamiento semántico: MAYOR.MENOR.PARCHÉ
SHA	Secure Hash Algorithm — algoritmo de hash criptográfico (SHA256 en el estándar)
SHA256	Variante de SHA con salida de 256 bits — usada para verificar integridad de artefactos
SLA	Service Level Agreement — acuerdo contractual de nivel de servicio
SLI	Service Level Indicator — métrica concreta medible de calidad del servicio
SLO	Service Level Objective — objetivo interno sobre un SLI
SRE	Site Reliability Engineering — ingeniería de confiabilidad del sitio
SRI	Servicio de Rentas Internas — institución propietaria del estándar
SSO	Single Sign-On — autenticación única centralizada vía Keycloak
TLS	Transport Layer Security — protocolo de seguridad de capa de transporte
URI	Uniform Resource Identifier — identificador uniforme de recurso
URL	Uniform Resource Locator — localizador uniforme de recurso
UUID	Universally Unique Identifier — identificador único universal (v4 en X-Correlation-Id)
VPN	Virtual Private Network — red privada virtual
WAF	Web Application Firewall — cortafuegos de aplicaciones web
YAML	YAML Ain't Markup Language — formato de serialización legible por humanos

b. Definiciones

Término	Definición
410 Gone	Código HTTP (RFC 9110) que indica eliminación permanente de un recurso. Obligatorio para APIs en estado retired. Nunca usar 404 ni 503. El body debe incluir migration_url.
API	Contrato de interacción entre un consumidor y un proveedor que expone capacidades o datos de manera controlada, versionada y gobernable.
API Gateway	Componente de infraestructura que actúa como punto único de entrada para tráfico externo, aplicando políticas de autenticación, autorización, rate limiting, ruteo y observabilidad antes de enrutar las solicitudes al backend.
API Owner	Persona o equipo responsable funcional y técnico de un Servicio de API: define propósito, scopes, consumidores y aprueba cambios de contrato o exposición. No opera el Gateway directamente.
API Product	Unidad lógica de gobierno que agrupa una o más APIs de un dominio bajo un identificador estable (productId). Tiene ownership, criticidad, planes de consumo y scopes definidos.
Artefacto inmutable	Paquete de software (imagen de contenedor, binario) que no se recompila entre ambientes. Solo cambia la configuración externa. Garantiza que lo que se prueba en PREPROD es lo que se despliega en PROD.
AsyncAPI	Especificación de contrato para APIs orientadas a eventos (Kafka, AMQP, WebSocket), análoga a OpenAPI para REST. Versión 3.x es la referencia actual.
Backend no internet-facing	Principio de frontera de confianza: un servicio backend no debe ser accesible directamente desde internet. Todo tráfico externo pasa por el API Gateway.
Breaking change	Modificación en el contrato de una API que rompe la compatibilidad con consumidores existentes. Requiere incremento de versión mayor y ventana de deprecación.
Canary deployment	Estrategia de despliegue progresivo donde una nueva versión recibe un porcentaje incremental de tráfico real antes del corte total.
Catálogo / Registro	Inventario institucional (source of truth) de APIs, productos, consumidores, planes, evidencias, estados y excepciones. En el estándar APIM v2.0 es el directorio api-catalog/ en Git.
Client Credentials Flow	Flujo OAuth 2.0 para autenticación M2M donde una aplicación cliente obtiene tokens usando client_id + client_secret sin intervención de usuario final.
Consumer / Consumidor	Aplicación, sistema o partner que consume un API Product. Se identifica con un consumerId único y está asociado a credenciales y suscripciones a planes.
Contract-First	Enfoque de desarrollo donde se define y aprueba el contrato OpenAPI antes de implementar el backend. Garantiza acuerdos claros proveedor/consumidor y reduce retrabajo.
Correlation ID	Identificador único (UUID v4) propagado en cada solicitud a lo largo del flujo completo (Gateway → Backend → Logs). Facilita diagnóstico y reduce MTTR.
Criticidad	Clasificación del impacto institucional de una API. Alta: impacto crítico en operación o dato sensible. Media: impacto operativo significativo. Baja: impacto menor. Determina controles mínimos, timeouts y plazos de deprecación.
Deprecación	Proceso formal de anuncio del retiro planificado de una versión o API, con ventana de convivencia (mínimo 30/60/90 días según criticidad), cabeceras RFC 8594 + 9745, y ruta de migración.
Drift	Diferencia entre la configuración declarada en el repositorio (Policies as Code) y la configuración efectivamente aplicada en el Gateway. Debe detectarse y corregirse automáticamente.

Event-Driven API	API basada en intercambio de mensajes o eventos asíncronos (Kafka, AMQP, etc.) en lugar de solicitudes síncronas HTTP. Requiere contrato AsyncAPI.
Gate (Quality Gate)	Control automático en el pipeline CI/CD que bloquea la promoción si no se cumplen criterios mínimos de calidad, seguridad o gobierno. Devuelve exit 0 (PASS) o exit 1 (FAIL).
Gateway Policies as Code	Práctica de gestionar toda la configuración del API Gateway como código en repositorios versionados, con el mismo rigor que el software. Elimina configuraciones manuales en producción.
Idempotencia	Propiedad de una operación donde múltiples ejecuciones con la misma entrada producen el mismo resultado sin efectos secundarios adicionales. Obligatoria para operaciones con retries.
JWT	Token de autenticación/autorización firmado digitalmente, compuesto por header, payload (claims: sub, iss, aud, exp, scopes) y firma. El Gateway valida la firma y los claims antes de enrutar.
Keycloak / Red Hat SSO	Plataforma de gestión de identidad y acceso, elegida como IdP institucional del SRI. Emite y valida tokens OIDC/OAuth 2.0. Implementado en el realm institucional.
mTLS	Extensión de TLS donde ambas partes (cliente y servidor) se autentican mediante certificados digitales. Recomendado para integraciones críticas M2M o con datos sensibles.
OAuth 2.0	Marco de autorización (RFC 6749) que permite a una aplicación obtener acceso limitado a recursos de otro servicio usando tokens de acceso. Base del Client Credentials Flow para M2M.
OIDC	Capa de identidad sobre OAuth 2.0 (OpenID Connect) que añade autenticación estandarizada con tokens ID. Base del SSO institucional del SRI sobre Keycloak.
OpenAPI Specification	Estándar (v3.1) para describir APIs HTTP/REST en formato YAML o JSON. Define endpoints, métodos, parámetros, esquemas, seguridad y respuestas. El contrato se valida en el stage lint.
OpenTelemetry	Marco de observabilidad de la CNCF que unifica trazas distribuidas, métricas y logs bajo un estándar agnóstico de herramienta. Referencia para propagación de X-Correlation-Id.
Plan	Paquete de acceso a un API Product para un consumidor. Define: scopes permitidos, rate limit (req/min), quota diaria y vigencia. Declarado en plans/{planId}.yaml (Anexo B).
Policies as Code	Ver Gateway Policies as Code. Toda configuración de rutas, planes, CORS, rate limits y headers mantenida como YAML en repositorio versionado.
productId	Identificador único, estable, en kebab-case del API Product. Nunca incluye versión ni ambiente. Regex: <code>^[a-z][a-z0-9-]+-(rest events graphql)\$</code> . No cambia tras publicación.
Quota	Límite de consumo acumulado en un periodo (diario o mensual). Complementa el rate limit. Ejemplo: 20.000 solicitudes/día. HTTP 429 al exceder.
Rate limit	Límite de tasa de solicitudes en una ventana temporal corta (por minuto). Protege el backend de picos de tráfico. HTTP 429 Too Many Requests al exceder, con cabecera Retry-After.
Release tag	Etiqueta inmutable en el repositorio de código (git tag) que identifica una versión específica. Habilita trazabilidad y rollback. El campo contract_url debe apuntar siempre a un tag.
Rollback	Proceso de revertir un despliegue a una versión anterior estable ante una falla o degradación. Requiere ticket y se ejecuta via pipeline.
Runbook	Documento operativo de máximo 2 páginas por API. Contiene: owner, endpoint PROD, consumidores activos, pasos de contención, rollback y escalación. Requerido antes de la primera promoción a PROD.
Scope	Permiso granular que habilita una acción específica sobre un API Product. Convención: <code>api:{productId}:{accion}</code> . Regex: <code>^api:[a-z][a-z0-9-]+:[a-z]+\$</code> . Asignado por mínimo privilegio.

Secret Manager / Vault	Herramienta para almacenar, rotar, auditar y distribuir secretos (client_secrets, certificados) fuera del código fuente. El campo secret_ref del Anexo C-2 referencia la ruta en Vault.
Servicio de API	Unidad gobernada en el catálogo que representa una API REST o de eventos del mismo dominio. Tiene estado de ciclo de vida: draft → published → deprecated → retired.
SLA	Compromiso contractual formal (generalmente con terceros) sobre niveles de servicio. Se apoya en SLOs internos medibles.
SLI	Métrica concreta y medible que representa un aspecto de la calidad del servicio. Ejemplos: latencia p95, tasa de éxito, disponibilidad.
SLO	Objetivo interno sobre un SLI. Ejemplo: disponibilidad ≥ 99,5%. Base para alertas y priorización operativa. Diferente del SLA (compromiso contractual).
sunset_date	Fecha de retiro efectivo de una API declarada en el YAML de registro (Anexo C) al entrar en estado deprecated. El Gateway inyecta esta fecha en la cabecera HTTP Sunset (RFC 8594).
Suscripción	Vínculo formal entre consumerId, planId y productId con estado y vigencia. Es la unidad operativa de acceso del consumidor. Máximo 12 meses para externos y partners.
Tenant	Unidad de segregación lógica en la plataforma (organización, área, dominio) que agrupa productos, consumidores y planes con aislamiento de configuración.
TLS	Protocolo criptográfico para cifrado de comunicaciones en tránsito. Versión mínima: TLS 1.2. Objetivo en PROD: TLS 1.3 (criticidad Alta).
Waiver	Excepción controlada, aprobada y documentada a una regla del estándar. Vigencia máxima 365 días. Requiere controles compensatorios verificables y fecha objetivo de regularización. Aprobado por ARQ.
Wildcard (*)	Carácter comodín. Prohibido en: (1) allowed_origins del cors.yaml — el gate lo detecta y bloquea con exit 1. (2) scopes — no se concede acceso genérico sin justificación de Seguridad.

7. Bibliografía

- AsyncAPI Initiative. (2024). AsyncAPI Specification (Version 3.0.0). <https://www.asyncapi.com/docs/reference/specification/v3.0.0>
- International Organization for Standardization. (2022). ISO/IEC 27001:2022 — Information security management systems. ISO.
- OpenAPI Initiative. (2024). OpenAPI Specification (Version 3.1). <https://spec.openapis.org/oas/latest.html>
- OpenID Foundation. (2014). OpenID Connect Core 1.0. https://openid.net/specs/openid-connect-core-1_0.html
- OpenTelemetry Authors. (2024). OpenTelemetry Specification. <https://opentelemetry.io/docs/specs/>
- OWASP Foundation. (2023). OWASP API Security Top 10 — 2023. <https://owasp.org/API-Security/editions/2023/en/0x00-header/> [Relevante para §6 Seguridad — alineación de controles con los 10 riesgos principales de APIs]
- NIST. (2021). NIST SP 800-204A — Building Secure Microservices-based Applications Using Service-Mesh Architecture. NIST. <https://csrc.nist.gov/publications/detail/sp/800-204a/final> [Relevante para §6 mTLS y autenticación M2M]
- Richardson, L. & Amundsen, M. (2013). RESTful Web APIs. O'Reilly Media. [Referencia general de diseño REST]

- CNCF. (2024). OpenTelemetry — Trace Context Propagation. <https://opentelemetry.io/docs/concepts/context-propagation/> [Relevante para §7 — propagación de trace-id/correlation-id]
- Newman, S. (2019). Monolith to Microservices. O'Reilly Media. [Contexto para gestión de APIs en arquitecturas de microservicios]
- Sporny, M. & Longley, D. (2022). Verifiable Credentials Data Model (W3C Recommendation). <https://www.w3.org/TR/vc-data-model/> [Contexto futuro para identidad descentralizada]

8. Anexos

Anexo A: Catálogo API - YAML de registro

Campo	Tipo	¿Obligatorio?	Descripción y restricciones
productId	string	SÍ	Identificador único del API Product. Regex: <code>^[a-z][a-z0-9-]+(- (rest events graphql))\$</code> . Patrón: {dominio}-{nombre-funcional}-{protocolo}. NO cambia nunca. Ej: pagos-notificacion-rest
domain	string enum	SÍ	Proceso de negocio. Debe ser un valor del Anexo F (taxonomy-domains.json). Lo define el API Owner. Ej: pagos
resource	string	SÍ	Recurso funcional principal que expone la API. Kebab-case. Usado para control de unicidad. Ej: notificacion-pago
exposure	enum	SÍ	Tipo de exposición: internal external partner
auth	enum	SÍ	Mecanismo de autenticación: oidc (Red Hat SSO) oauth2-cc mtls none
criticality	enum	SÍ	Criticidad del servicio: Alta Media Baja. Determina los plazos de deprecación (R-07) y los planes de uso recomendados (Anexo B).
owner_name	string	SÍ	Nombre completo del API Owner funcional (responsable del proceso de negocio)
owner_email	email	SÍ	Email institucional del API Owner. Regex: <code>^[\w.+-]+@sri\.gob\.ec\$</code>
owner_area	string	SÍ	Área o departamento del API Owner. Ej: Dirección de Sistemas de Pago
tech_name	string	SÍ	Nombre del responsable técnico del microservicio. Puede ser la misma persona que el owner_name.
tech_email	email	SÍ	Email institucional del responsable técnico. Regex: <code>^[\w.+-]+@sri\.gob\.ec\$</code>
state	enum	SÍ	Estado del ciclo de vida: draft published deprecated retired. Inicia siempre en draft. Solo el pipeline actualiza a published.
contract_url	url	SÍ en published	URL al contrato OpenAPI 3.1 en GitLab, apuntando a un tag inmutable (/tags/v1.2.0/). NUNCA a main o develop.

pipeline_url	url	Sí en published	URL al último pipeline exitoso en GitLab CI que desplegó esta versión en PROD.
dashboard_url	url	Sí en published	URL al dashboard de observabilidad (Grafana, ELK, etc.) con métricas de esta API.
consumers	array	Recomendado	Lista de consumerId de los consumidores autorizados. Cada entry referencia un YAML en api-catalog/consumers/. Para el control de unicidad del gate, si está vacío y exposure=internal y auth=none, la API puede no requerir catálogo (ver R-11).
sunset_date	date ISO	Sí en deprecated	Fecha de retiro efectivo. Formato: YYYY-MM-DD. El Gateway inyecta esta fecha en la cabecera Sunset (RFC 8594) al pasar a deprecated.
deprecation_reason	string	Sí en deprecated	Motivo de la deprecación. Ej: Sustituida por pagos-notificacion-v2-rest
migration_url	url	Recomendado en deprecated	URL del API que reemplaza a esta. Se incluye en la cabecera Link y en el body del 410 Gone.
description	string	Recomendado	Descripción funcional de la API en lenguaje de negocio (máx 500 caracteres).
tags	array string	Opcional	Etiquetas de búsqueda en el catálogo. Ej: [pago, notificacion, externo]

Anexo B: Registro de Consumidor – YAML Registro

Campo	Tipo	Obligatorio	Descripción y restricciones
consumerId	string	OBLIGATORIO	Identificador único. Kebab-case. Patrón: {institucion}-{sistema}. Regex: ^[a-z][a-z0-9-]+\$. Nunca reutilizable. Ej: banco-amarillo-sistemas · No incluir ambiente ni versión
name	string	OBLIGATORIO	Nombre legible del sistema o entidad consumidora. Ej: Banco Amarillo — Sistema de Integración
type	enum	OBLIGATORIO	Relación del consumidor con la institución. Valores: interno externo partner
client_type	enum	OBLIGATORIO	Tipo técnico del cliente. Determina si aplica política CORS. Valores: browser server mobile browser → CORS requerido con allowed_origins explícita en cors.yaml server / mobile → CORS no aplica · Sin este campo el gate bloquea el deploy
contact_name	string	OBLIGATORIO	Nombre del responsable técnico del consumidor (contacto para renovación y deprecación).

contact_email	email	OBLIGATORIO	Correo del responsable del consumidor. Para externos: cualquier dominio válido.
keycloak_client_id	string	OBLIGATORIO	ID del client en Red Hat SSO. Uno por consumidor — nunca compartido entre consumidores. Ej: banco-amarillo-sistemas-prod · El gate verifica que no esté asignado a otro consumidor
plan_id	string	OBLIGATORIO	Identificador del plan de uso asignado. Referencia un archivo en plans/. Ej: plan-partner · Ver Anexo B para valores de cada plan
scopes	array	OBLIGATORIO	Scopes concedidos a este consumidor. Solo los estrictamente necesarios (mínimo privilegio). Convención: api:{productId}:{accion} · Regex: ^api:[a-z][a-z0-9-]*:[a-z]+\$ ✓ api:pagos-notificacion:read · X api:pagos-notificacion:* · X api:pagos-notificacion:admin (sin aprobación SEC)
expires	date ISO	OBLIGATORIO si type: externo o partner	Fecha de vencimiento. Máximo 12 meses desde la creación. Formato: YYYY-MM-DD · A los -30 días: alerta automática de renovación Sin renovación: 401 subscription_expired
secret_ref	string	RECOMENDADO	Referencia al secret en el gestor de secretos. Nunca el secret en texto plano. Ej: vault:secret/apim/banco-amarillo-sistemas/client_secret
approved_by	email	RECOMENDADO	Email del API Owner que aprobó la suscripción. Ej: mlopez@sri.gob.ec
approval_ticket	string	RECOMENDADO	Número de ticket de aprobación formal. Ej: TCK-2026-00241
description	string	OPCIONAL	Propósito de integración en lenguaje de negocio. Máx. 300 caracteres.

Anexo C: Planes de Uso (Rate Limit y quota) – YAML

Campo	Tipo	Obligatorio	Descripción
planId	string	OBLIGATORIO	Identificador del plan. Kebab-case. Debe coincidir con el nombre del archivo. Ej: plan-partner
name	string	OBLIGATORIO	Nombre legible del plan.
rate_limit_rpm	integer	OBLIGATORIO	Límite de solicitudes por minuto por consumidor. HTTP al exceder: 429 con Retry-After.

quota_daily	integer null	OBLIGATORIO	Límite diario acumulado. null = sin límite diario.
target_audience	string	RECOMENDADO	A qué tipo de consumidor está dirigido este plan.
description	string	RECOMENDADO	Descripción del propósito del plan.

Anexo D: Configuración Gateway por API – YAML

Campo	Tipo	Obligatorio	Descripción y restricciones
productId	string	OBLIGATORIO	Debe coincidir exactamente con el productId del Anexo C. Usado como clave de asociación.
upstream_url	url	OBLIGATORIO	URL interna del microservicio backend en Kubernetes. Ej: http://sri-pagos-notificacion-service.pagos.svc.cluster.local:8080 El ambiente (testing/preprod/prod) se obtiene del archivo env/{ambiente}.yaml
path_prefix	string	OBLIGATORIO	Prefijo de ruta expuesto por el Gateway. Ej: /pagos-notificacion/v1 · No puede ser /health, /actuador/** ni /metrics
strip_path_prefix	boolean	OBLIGATORIO	Si true, el Gateway elimina el path_prefix antes de enrutar al backend. Ej: true → el backend recibe /v1/notificaciones no /pagos-notificacion/v1/notificaciones
connect_timeout	integer (s)	OBLIGATORIO	Timeout para establecer conexión con el backend. HTTP al exceder: 504. Baseline por criticidad (Anexo B): Alta=5s · Media=10s · Baja=15s
read_timeout	integer (s)	OBLIGATORIO	Timeout para recibir respuesta completa del backend. HTTP al exceder: 504. Baseline: Alta=15s · Media=30s · Baja=60s
max_request_size	string	OBLIGATORIO	Tamaño máximo del payload de la solicitud. HTTP al exceder: 413. Baseline: Alta=1MB · Media=5MB · Baja=10MB · Formato: 10MB
max_response_size	string	OBLIGATORIO	Tamaño máximo del payload de la respuesta. HTTP al exceder: 502. Baseline: Alta=5MB · Media=10MB · Baja=20MB
retries	integer	RECOMENDADO	Número de reintentos ante fallo de backend. Solo para operaciones idempotentes. Ej: 2 · Por defecto: 0 para operaciones POST no idempotentes
retry_on	array		Códigos HTTP que activan reintento.

		RECOMENDADO si retries > 0	Ej: [502, 503, 504]
--	--	-------------------------------	---------------------

Anexo E. Configuración CORS por API – YAML

Campo	Tipo	Obligatorio	Descripción y restricciones
productId	string	OBLIGATORIO	Debe coincidir con el productId del Anexo C.
enabled	boolean	OBLIGATORIO	true solo si hay consumidores client_type: browser. false en todos los demás casos. El gate rechaza enabled: true sin allowed_origins definido
allowed_origins	array url	OBLIGATORIO si enabled: true	Lista de orígenes permitidos. URLs completas con protocolo y dominio. ✓ ["https://portal.sri.gob.ec"] X ["*"] — el gate detecta wildcards con regex y bloquea exit 1 X ["http://portal.sri.gob.ec"] — HTTP sin TLS rechazado en PROD
allowed_methods	array	OBLIGATORIO si enabled: true	Métodos HTTP permitidos. Solo los que usa la API · Ej: [GET, POST, OPTIONS]
allowed_headers	array	OBLIGATORIO si enabled: true	Headers que puede enviar el cliente. Ej: [Authorization, Content-Type, X-Correlation-Id]
max_age	integer (s)	RECOMENDADO	Tiempo en segundos que el navegador puede cachear la respuesta preflight. Ej: 3600